

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-25

Заседатель Н.С.

Проверила:

Хацкевич А.С.

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Ход работы:

Задание 1. Базовый перехват запуска процессов

Разработана программа на Python с использованием библиотеки BCC. В ядре Linux используется eBPF-программа, подключённая к tracepoint `sys_enter_execve`. При каждом вызове `execve` извлекаются PID процесса, UID пользователя и имя запускаемого файла.

```
#!/usr/bin/python3
from bcc import BPF

bpf_text = """
#include <uapi/linux/ptrace.h>

struct data_t {
    u32 pid;
    u32 uid;
    char comm[256];
};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_execve) {
    struct data_t data = {};

    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid();

    bpf_probe_read_user_str(&data.comm, sizeof(data.comm),
                           args->filename);

    events.perf_submit(args, &data, sizeof(data));

    return 0;
}
"""

b = BPF(text=bpf_text)

print("[*] Мониторинг запущен. Ожидание вызовов execve...")

def print_event(cpu, data, size):
    event = b["events"].event(data)
```

```

print(f'PID: {event.pid}, UID: {event.uid}, "
      f"Процесс: {event.comm.decode('utf-8', 'replace')}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        break

```

Программа запускается с правами суперпользователя:

sudo python3 monitor_template.py

```

connecting_software@User:~/lab3$ sudo python3 monitor_template.py
In file included from <built-in>:2:
In file included from /virtual/include/bcc/bpf.h:12:
In file included from include/linux/types.h:6:
In file included from include/uapi/linux/types.h:14:
In file included from include/uapi/linux/posix_types.h:5:
In file included from include/linux/stddef.h:5:
In file included from include/uapi/linux/stddef.h:6:
In file included from include/linux/compiler_types.h:120:
include/linux/compiler-clang.h:46:9: warning: '__HAVE_BUILTIN_BSWAP
#define __HAVE_BUILTIN_BSWAP32__

```

Для проверки работы мониторинга во втором терминале были выполнены команды **cat** и **ls**.

```

connecting_software@User:~/lab3$ cat /etc/hostname
User
connecting_software@User:~/lab3$ ls
monitor_template.py

```

В результате запуска команд монитор зафиксировал соответствующие вызовы **execve**:

```

6 warnings generated.
[*] Мониторинг запущен. Ожидание вызовов execve...
PID: 1459, UID: 1000, Процесс: /usr/bin/cat
PID: 1460, UID: 1000, Процесс: /usr/bin/ls

```

Задание 2. Динамический анализ аномального поведения (Индивидуальный вариант)

Целевой системный вызов и условие аномального поведения

Согласно индивидуальному варианту 8 в качестве целевого системного вызова выбран **sys_enter_renameat2**. Аномальным поведением считается попытка переименования или сокрытия файла с расширением **.log**. Поэтому eBPF-программа проверяет старое и новое имя файла. Если хотя бы одно из них заканчивается на **.log**, формируется событие для передачи в пространство пользователя.

eBPF-зонд

Для перехвата системного вызова используется **tracepoint sys_enter_renameat2**. В ядре извлекаются следующие данные:

- PID процесса;
- UID пользователя;

- старое имя файла oldname;
- новое имя файла newname.

Для хранения данных используется BPF_PERCPU_ARRAY, что позволяет избежать превышения ограничения размера стека eBPF.

Листинг программы:

```
#!/usr/bin/python3
from bcc import BPF

bpf_text = """
#include <uapi/linux/ptrace.h>

struct data_t {
    u32 pid;
    u32 uid;
    char oldname[256];
    char newname[256];
};

BPF_PERF_OUTPUT(events);

BPF_PERCPU_ARRAY(temp, struct data_t, 1);

TRACEPOINT_PROBE(syscalls, sys_enter_renameat2)
{
    u32 key = 0;
    struct data_t *data = temp.lookup(&key);

    if (!data)
        return 0;

    data->pid = bpf_get_current_pid_tgid() >> 32;
    data->uid = bpf_get_current_uid_gid();

    bpf_probe_read_user_str(&data->oldname,
                            sizeof(data->oldname),
                            args->oldname);

    bpf_probe_read_user_str(&data->newname,
                            sizeof(data->newname),
                            args->newname);

    int old_log = 0;
    int new_log = 0;

    for (int i = 0; i < 252; i++) {
        if (data->oldname[i] == '\\0')
            break;
```

```

    if (data->oldname[i] == '.' &&
        data->oldname[i + 1] == 'l' &&
        data->oldname[i + 2] == 'o' &&
        data->oldname[i + 3] == 'g' &&
        data->oldname[i + 4] == '\\0') {
        old_log = 1;
        break;
    }
}

for (int i = 0; i < 252; i++) {
    if (data->newname[i] == '\\0')
        break;

    if (data->newname[i] == '.' &&
        data->newname[i + 1] == 'l' &&
        data->newname[i + 2] == 'o' &&
        data->newname[i + 3] == 'g' &&
        data->newname[i + 4] == '\\0') {
        new_log = 1;
        break;
    }
}

if (old_log || new_log) {
    events.perf_submit(args, data, sizeof(*data));
}

return 0;
}
""""

b = BPF(text=bpf_text)

print("[*] Мониторинг renameat2 запущен...")
print("[*] Ожидание попыток переименования файлов .log...")

def print_event(cpu, data, size):
    event = b["events"].event(data)

    oldname = event.oldname.decode("utf-8", "replace")
    newname = event.newname.decode("utf-8", "replace")

    print(f"[ALERT] PID: {event.pid}, UID: {event.uid}")
    print(f"    Переименование: {oldname} -> {newname}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()

```

```
except KeyboardInterrupt:
    print("\n[*] Завершение работы монитора.")
    break
```

Фильтрация событий на стороне ядра

Фильтрация выполняется непосредственно внутри eBPF-программы. Событие передаётся через **BPF_PERF_OUTPUT** только при выполнении условия:

```
if (old_log || new_log) { events.perf_submit(args, data, sizeof(*data)); }
```

Обычные вызовы `renameat2`, не связанные с файлами `.log`, не передаются в Python-скрипт. Это позволяет выполнять фильтрацию на стороне ядра и не передавать в пространство пользователя все подряд события системного вызова.

Генерация и перехват аномального поведения

Для проверки работы зонда был создан файл **test.log** и выполнено его переименование:

```
connecting_software@User:~/lab3$ echo "test" > test.log
connecting_software@User:~/lab3$ mv test.log hidden.txt
```

В результате **eBPF**-зонд обнаружил попытку переименования файла `.log` и сформировал предупреждение:

```
connecting_software@User:~/lab3$ sudo python3 task2.py
In file included from <built-in>:2:
In file included from /virtual/include/bcc/bpf.h:12:
In file included from include/linux/types.h:6:
In file included from include/uapi/linux/types.h:14:
In file included from include/uapi/linux/posix_types.h:5:
In file included from include/linux/stddef.h:5:
In file included from include/uapi/linux/stddef.h:6:
In file included from include/linux/compiler_types.h:120:
6 warnings generated.
[*] Мониторинг renameat2 запущен...
[*] Ожидание попыток переименования файлов .log...
[ALERT] PID: 1524, UID: 1000
        Переименование: test.log -> hidden.txt
|
```

Дополнительно была выполнена проверка обычного переименования:

```
connecting_software@User:~/lab3$ echo "test" > normal.txt
connecting_software@User:~/lab3$ mv normal.txt normal2.txt
```

При этом предупреждение **[ALERT]** не появилось, что подтверждает корректную работу фильтрации на стороне ядра.

Задание 3. Интеграция с пространством пользователя и аудит

Python-скрипт из задания 2 был расширен функцией автоматической записи обнаруженных аномалий в локальный файл аудита `security_audit.log`. При получении события от eBPF-программы Python-скрипт формирует запись, содержащую временную метку, PID и UID процесса, а также информацию о переименовании файла.

Листинг программы:

```
#!/usr/bin/python3
from bcc import BPF
```

```
from datetime import datetime
```

```
bpf_text = ""
```

```
#include <uapi/linux/ptrace.h>
```

```
struct data_t {  
    u32 pid;  
    u32 uid;  
    char oldname[256];  
    char newname[256];  
};
```

```
BPF_PERF_OUTPUT(events);
```

```
BPF_PERCPU_ARRAY(temp, struct data_t, 1);
```

```
TRACEPOINT_PROBE(syscalls, sys_enter_renameat2)
```

```
{  
    u32 key = 0;  
    struct data_t *data = temp.lookup(&key);  
  
    if (!data)  
        return 0;  
  
    data->pid = bpf_get_current_pid_tgid() >> 32;  
    data->uid = bpf_get_current_uid_gid();  
  
    bpf_probe_read_user_str(&data->oldname,  
                            sizeof(data->oldname),  
                            args->oldname);  
  
    bpf_probe_read_user_str(&data->newname,  
                            sizeof(data->newname),  
                            args->newname);  
  
    int old_log = 0;  
    int new_log = 0;  
  
    for (int i = 0; i < 252; i++) {  
        if (data->oldname[i] == '\\0')  
            break;  
  
        if (data->oldname[i] == '.' &&  
            data->oldname[i + 1] == 'l' &&  
            data->oldname[i + 2] == 'o' &&  
            data->oldname[i + 3] == 'g' &&  
            data->oldname[i + 4] == '\\0') {  
            old_log = 1;  
            break;  
        }  
    }  
}
```

```

for (int i = 0; i < 252; i++) {
    if (data->newname[i] == '\\0')
        break;

    if (data->newname[i] == '.' &&
        data->newname[i + 1] == 'l' &&
        data->newname[i + 2] == 'o' &&
        data->newname[i + 3] == 'g' &&
        data->newname[i + 4] == '\\0') {
        new_log = 1;
        break;
    }
}

if (old_log || new_log) {
    events.perf_submit(args, data, sizeof(*data));
}

return 0;
}
"""

```

```

b = BPF(text=bpf_text)

```

```

print("[*] Мониторинг renameat2 запущен...")
print("[*] Аудит сохраняется в security_audit.log")

```

```

def print_event(cpu, data, size):
    event = b["events"].event(data)

```

```

    oldname = event.oldname.decode("utf-8", "replace")
    newname = event.newname.decode("utf-8", "replace")

```

```

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```

    message = (
        f'{timestamp} | '
        f'PID: {event.pid} | '
        f'UID: {event.uid} | '
        f'Переименование: {oldname} -> {newname}'
    )

```

```

    print("[ALERT]")
    print(message)

```

```

    with open("security_audit.log", "a") as log:
        log.write(message + "\n")

```

```

b["events"].open_perf_buffer(print_event)

```

```
while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        break
```

Для проверки была создана аномальная ситуация — файл **audit_test.log** был переименован в **hidden_audit.txt**:

```
connecting_software@User:~/lab3$ echo "test" > audit_test.log
connecting_software@User:~/lab3$ mv audit_test.log hidden_audit.txt
```

После обнаружения события программа вывела предупреждение:

```
6 warnings generated.
[*] Мониторинг renameat2 запущен...
[*] Аудит сохраняется в security_audit.log
[ALERT]
2026-10-06 14:43:19 | PID: 1977 | UID: 1000 | Переименование: audit_test.log -> hidden_audit.txt
```

Одновременно информация была автоматически записана в файл **security_audit.log**.

Содержимое файла аудита:

```
connecting_software@User:~/lab3$ cat ~/lab3/security_audit.log
2026-10-06 14:43:19 | PID: 1977 | UID: 1000 | Переименование: audit_test.log -> hidden_audit.txt
```

Реализована интеграция eBPF-программы с пространством пользователя: обнаруженное в ядре аномальное событие передаётся Python-скрипту, после чего автоматически сохраняется в локальный журнал безопасности **security_audit.log**.

Задание 4. Проектирование профилей защиты (Без написания кода)

Сравнение Seccomp-BPF и eBPF (kprobes)

Seccomp-BPF и обычный **eBPF**-зонд решают разные задачи.

Seccomp-BPF

Предназначен для ограничения поведения процесса. С его помощью можно задать фильтр системных вызовов, которые разрешены или запрещены процессу. Например, можно разрешить только необходимые системные вызовы, а попытку выполнения запрещённого вызова заблокировать или завершить процесс.

eBPF с kprobes/tracepoints

В основном используется для наблюдения за работой ядра. С помощью eBPF можно перехватывать события, получать информацию о процессах и системных вызовах, анализировать параметры операций и передавать результаты в пространство пользователя.

В лабораторной работе eBPF использовался для мониторинга **sys_enter_renameat2**. Программа обнаруживала попытки переименования файлов с расширением **.log** и передавала информацию в Python-скрипт. При этом сама операция переименования не блокировалась.

Характеристика	Seccomp-BPF	eBPF
Основное назначение	Ограничение системных вызовов	Мониторинг и анализ
Контроль системных вызовов	Да	Да, в основном наблюдение
Блокировка операции	Да, через заданные действия seccomp	Обычно нет
Анализ параметров	Ограниченный	Более гибкий
Использование в лабораторной	Не использовался	Использовался для обнаружения аномалий

Стандартный eBPF-зонд преимущественно используется для мониторинга, обнаружения и аудита, а не для блокировки действий вредоносного ПО. eBPF-программа может обнаружить подозрительное действие и сообщить о нём, но для непосредственного ограничения системных вызовов процесса существуют специализированные механизмы, например Seccomp-BPF.

Атака TOCTOU применительно к eBPF-зонду

TOCTOU (Time-of-Check to Time-of-Use) – это атака типа «время проверки – время использования». Она возникает, когда программа сначала проверяет состояние объекта, а затем через некоторое время использует этот объект. Между проверкой и использованием злоумышленник может изменить объект. Применительно к разработанному eBPF-зонду проблема может возникнуть следующим образом. Зонд получает информацию о `renameat2` и проверяет имя файла. Например, обнаруживается операция с файлом `test.log`. Если после проверки состояние файла или его имя изменяется другим процессом, фактическая операция может отличаться от той, которую анализировал зонд. Особенно важно, что наш зонд предназначен для наблюдения, а не для принятия решения о блокировке операции. Поэтому между моментом обнаружения события и реакцией Python-скрипта может пройти некоторое время. За этот промежуток состояние файла уже может измениться. Для противодействия TOCTOU необходимо не разделять проверку и критическую операцию, а выполнять проверку и принятие решения атомарно в ядре. Для операций с файлами можно использовать атомарные системные механизмы и ограничения доступа, а для контроля системных вызовов – применять Seccomp-BPF или другой механизм принудительного контроля. Для рассматриваемой архитектуры рациональным вариантом является сочетание механизмов:

eBPF – обнаружение и аудит – система реагирования

и

Seccomp/механизмы контроля доступа – предотвращение запрещённой операции.

eBPF отвечает за наблюдение и регистрацию событий, а механизм принудительного контроля используется непосредственно для предотвращения опасных действий.

Вывод: практически освоил методы разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracemarks) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).